

I genuinely think the project you've converged on is much stronger than the original idea. It no longer looks like a CRUD application with Kafka added—it now resembles a small event-driven fintech platform. If you implement it cleanly, it will be an excellent talking point for interviews at companies like Groww, Zerodha, PhonePe, Jupiter, Navi, CRED, Fi, Slice, and many backend teams working with financial products.

Project Overview

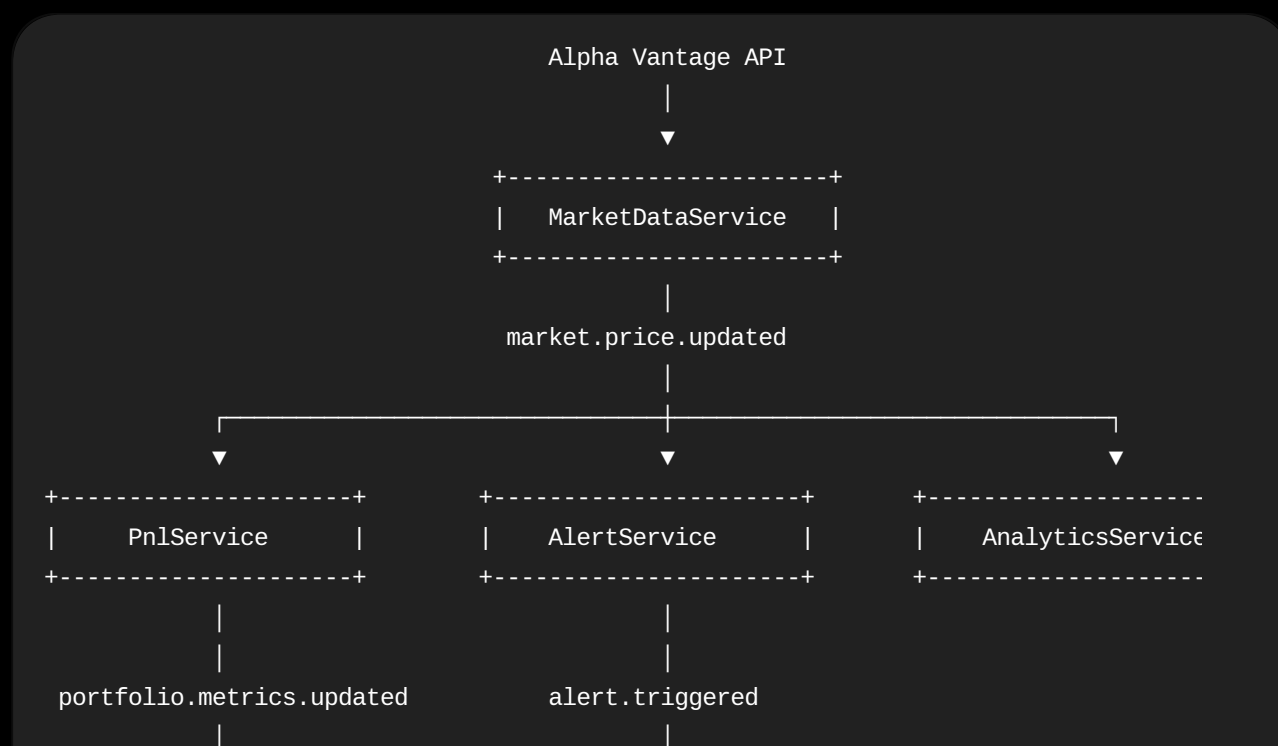
Wealth+ : Smart Portfolio Management & AI Robo Advisor Platform

A distributed event-driven backend platform where investors can:

- Register and create multiple portfolios
- Buy and sell stocks
- Track portfolio performance in near real time
- Receive intelligent portfolio alerts
- Get AI-powered portfolio rebalancing recommendations
- View complete order and alert history

Instead of one monolithic application, the platform is divided into multiple services communicating asynchronously through Kafka.

High Level Architecture





This architecture demonstrates:

- Event Driven Architecture
- CQRS Read Models
- Kafka Fan-out
- Asynchronous Communication
- Microservice Boundaries

Services

1. Portfolio Service

This is the core business service.

Responsibilities

- Investor Registration
- Portfolio Creation
- Buy Stocks
- Sell Stocks
- Holdings Management
- Order Management

Database Tables

Investor

Portfolio

Order

Holdings

APIs

POST /investors

POST /portfolios

POST /orders/buy

POST /orders/sell

GET /holdings

GET /orders

Events Published

portfolio.order.executed

Contains

Java

OrderExecutedEvent

For audit.

portfolio.holdings.updated

Contains

Java

HoldingUpdatedEvent

This is the canonical event.

Every downstream service updates its own read model.

Business Logic

Buying

```
BUY 10 AAPL @200
```

Updates

```
Orders
```

```
Holdings
```

Weighted average calculation

```
(OldQty × OldAvg + NewQty × NewPrice)  
/ TotalQty
```

Selling

Quantity decreases.

Average price remains unchanged.

2. MarketDataService

Responsible for collecting market prices.

Uses

```
Alpha Vantage API
```

Every 30 seconds

```
Java
```

```
@Scheduled
```

Calls

```
GLOBAL_QUOTE
```

Technologies

- Spring Scheduler
 - WebClient
 - Resilience4j
 - Redis
 - Kafka
-

Responsibilities

Fetch latest stock price.

Read previous price from Redis.

Compute

Current Price

Previous Price

Absolute Change

Percentage Change

Publish

`market.price.updated`

Redis

Stores

`price:AAPL`

instead of hitting Alpha Vantage repeatedly.

TTL

`30 seconds`

Resilience4j

Uses

- Circuit Breaker
- Retry
- Rate Limiter

to protect against API failures.

Event

Java

`MarketStockPriceEvent`

`symbol`

`previousPrice`

`currentPrice`

`absoluteChange`

`percentageChange`

`tradingDate`

`eventTime`

3. PnL Service

Responsible for calculating portfolio valuation.

This service never queries Portfolio Service.

Instead

Maintains its own read model.

Consumes

portfolio.holdings.updated

Updates

PortfolioHoldingSnapshot

Consumes

market.price.updated

Updates

latestMarketPrice

Recalculates metrics.

Stores

PortfolioHoldingSnapshot

PortfolioMetrics

Calculations

Current Value

$\Sigma(\text{quantity} \times \text{latestMarketPrice})$

Invested Amount

$\Sigma(\text{quantity} \times \text{averagePrice})$

PnL

Current Value - Invested Amount

PnL %

$(\text{PnL} / \text{InvestedAmount}) \times 100$

Publishes

portfolio.metrics.updated

Contains

Java

portfolioId

currentValue

investedAmount

pnl

pnlPercentage

updatedAt

4. Alert Service

One of the most interesting services.

Responsible for generating intelligent alerts.

Consumes

market.price.updated

Generates

Price Alerts

Examples

AAPL crossed ₹250

AAPL dropped below ₹180

AAPL moved >5%

Consumes

portfolio.metrics.updated

Generates

Portfolio Alerts

Examples

Portfolio profit exceeded 20%

Portfolio loss exceeded 10%

Portfolio crossed ₹10L value

Consumes

portfolio.holdings.updated

Maintains

AlertPortfolioHoldings

read model.

Tables

AlertConfiguration

AlertHistory

AlertPortfolioHoldings

Publishes

```
alert.triggered
```

Redis

Used for

Alert deduplication.

Without Redis

```
AAPL
```

```
251
```

```
252
```

```
253
```

```
254
```

User gets

```
4 notifications
```

Redis

```
alert:123
```

```
TTL 30 min
```

Avoids duplicate notifications.

5. Notification Service

Consumes

```
alert.triggered
```

Can support

- Email
- SMS
- Push Notification

Stores

Notification

NotificationHistory

For now

Just logging

would be sufficient.

6. Robo Advisor Service

Probably the strongest resume feature.

Consumes

portfolio.rebalance.triggered

Calls

Spring AI

Prompt

Investor Risk Profile

Moderate

Portfolio

AAPL 70%

MSFT 20%

Cash 10%

Generate a rebalance recommendation.

Returns

Reduce AAPL to 40%

Increase ETF allocation

Stores

Recommendation

Kafka Topics

market.price.updated

portfolio.order.executed

portfolio.holdings.updated

portfolio.metrics.updated

alert.triggered

portfolio.rebalance.triggered

Database Tables

Portfolio Service

Investor

Portfolio

Order

Holdings

PnL Service

PortfolioHoldingSnapshot

PortfolioMetrics

Alert Service

AlertConfiguration

AlertHistory

AlertPortfolioHoldings

Notification

Notification

NotificationHistory

AI

Recommendation

Redis

Use Cases

Market Price Cache

price:AAPL

Alert Deduplication

alert:123

Session Cache (optional)

Spring AI

Uses

OpenAI

Prompt

Risk Profile

Portfolio Allocation

PnL

Investment Horizon

Returns

AI generated recommendation.

Resilience4j

Protects

Alpha Vantage

Uses

Retry

3 retries

Circuit Breaker

Open

Half Open

Closed

Rate Limiter

Prevent exceeding Alpha Vantage free tier.

Design Patterns Used

- Event Driven Architecture
- CQRS (Read Models)

- Repository Pattern
 - Strategy Pattern (Alert Evaluation)
 - Builder Pattern
 - Factory Pattern (Event Builders)
 - Dependency Injection
 - Observer Pattern (Kafka Consumers)
-

Resume Talking Points

When you describe this project on your resume or in interviews, emphasize the engineering aspects rather than just the business features:

- Designed and implemented an event-driven microservices architecture using Spring Boot and Apache Kafka.
 - Built asynchronous workflows for portfolio management, real-time PnL calculation, and alert generation.
 - Integrated external market data with Resilience4j (Circuit Breaker, Retry, Rate Limiter) for fault tolerance.
 - Used Redis for low-latency market price caching and alert deduplication.
 - Implemented CQRS-style read models in downstream services to avoid cross-service database access.
 - Built AI-powered portfolio rebalancing recommendations using Spring AI.
 - Developed idempotent Kafka consumers with transactional processing to handle at-least-once delivery.
 - Designed scalable REST APIs, normalized database schemas, and event contracts for service communication.
-

Future Enhancements (If You Have Time)

These are the kinds of additions that elevate the project further and give you richer interview discussions:

- **Docker Compose** to orchestrate Kafka, Redis, PostgreSQL, and all services locally.
- **Prometheus + Grafana** with Micrometer metrics for monitoring event throughput, processing latency, and API health.
- **Distributed tracing** using OpenTelemetry and Zipkin/Jaeger.
- **Dead Letter Queue (DLQ)** for failed Kafka messages with retry and backoff strategies.

- **Transactional Outbox Pattern** to guarantee reliable event publishing without dual-write issues.
 - **Optimistic Locking** (`@Version`) on holdings to prevent lost updates during concurrent buy/sell operations.
 - **JWT Authentication & Role-Based Access Control** (Investor/Admin).
 - **Testcontainers** for integration tests involving PostgreSQL, Kafka, and Redis.
 - **CI/CD Pipeline** using GitHub Actions to run tests and build Docker images automatically.
 - **API Documentation** with OpenAPI/Swagger.
 - **Centralized Logging** using ELK/OpenSearch if you want to showcase observability.
-

Why this project stands out

Most resume projects are variations of CRUD applications with a message queue bolted on. This project demonstrates:

- Strong domain modeling (investments, portfolios, holdings, PnL).
- Event-driven communication between independent services.
- Proper ownership of data through service-specific read models.
- Fault tolerance with Resilience4j.
- Performance optimization with Redis.
- AI integration that solves a real business problem rather than being added for novelty.

That combination closely mirrors the kinds of backend systems built by modern fintech companies and provides plenty of depth for both coding and system design interviews.